MIT 8103 / 2026-2027

Portfolio 4: a simple document model

Use the same Green Basket Shop case study. Some products have weight; others have volume. A document can store the properties each product needs without adding a column for every possible property.

Model and implementation

The separate PDF 06 creates ProductDocuments and inserts two documents. ProductID identifies the product; Details contains JSON text. A JSON object is a collection of names and values. Here, brand is text and weightKg or volumeLitres is a number. Only T-SQL statements are executed; JSON inside a string is data, not another programming language.

```
-- Rice document stored in Details:
-- {"brand":"Farm Fresh","weightKg":25}
-- Oil document stored in Details:
-- {"brand":"Golden Oil","volumeLitres":5}
```

Query and change the documents

```
USE GreenBasketDB;
GO

-- Read one value from each document.
SELECT ProductID,
       JSON_VALUE(Details, '$.brand') AS Brand
FROM dbo.ProductDocuments
ORDER BY ProductID;

-- Find the rice document by its brand.
SELECT               ProductID,               Details
FROM                              dbo.ProductDocuments
WHERE JSON_VALUE(Details, '$.brand') = N'Farm Fresh';

-- Different documents can have different properties.
SELECT ProductID,
       JSON_VALUE(Details, '$.weightKg') AS WeightKg,
       JSON_VALUE(Details, '$.volumeLitres') AS VolumeLitres
FROM dbo.ProductDocuments
ORDER BY ProductID;

-- Change one property, then undo it for a repeatable lesson.
BEGIN TRANSACTION;
UPDATE dbo.ProductDocuments
SET Details = JSON_MODIFY(Details, '$.weightKg', 50)
WHERE ProductID = 1;

SELECT JSON_VALUE(Details, '$.weightKg') AS TrialWeight
FROM dbo.ProductDocuments WHERE ProductID = 1;
ROLLBACK TRANSACTION;

SELECT JSON_VALUE(Details, '$.weightKg') AS RestoredWeight
FROM dbo.ProductDocuments WHERE ProductID = 1;
```

Expected results

Brands: product 1 = Farm Fresh; product 3 = Golden Oil. The brand filter returns product 1. Product 1 has WeightKg = 25 and VolumeLitres = NULL; product 3 has WeightKg = NULL and VolumeLitres = 5. The trial weight is 50; after rollback it is 25. JSON_VALUE returns text, even when a stored value is numeric.

Why this model fits

Optional catalogue attributes vary by product. Documents keep those attributes together and allow new properties without changing the table definition. Relational tables are better for orders, quantities and prices because these need reliable relationships and constraints.

ISJSON validates JSON syntax only. Misspelled or missing properties are still possible. JSON_VALUE reads one simple value using a path: $.brand means the brand property at the root. JSON_MODIFY produces updated JSON text; UPDATE saves it. Numeric arithmetic would need a conversion from the text returned by JSON_VALUE.

Scope and evidence

This is a NoSQL-style document model implemented in a SQL Server table, not MongoDB or another dedicated NoSQL engine. It meets the request to use T-SQL only; the lecturer decides whether it satisfies the NoSQL implementation requirement. Save screenshots of the table and all results.